

SWK

Vorlesungsnotizen

Stand: 31. August 2026

Inhaltsverzeichnis

1 Software Konstruktion

Softwarekonstruktion ist ingenieurmäßiges Vorgehen bei der Erstellung von Software: planbar, überprüfbar und wiederholbar statt handwerklich und personenabhängig.

Definition

Ingenieurmäßiges Vorgehen heißt: Anforderungen werden erhoben und spezifiziert, eine Lösung wird entworfen und dokumentiert, die Umsetzung wird gemessen und geprüft, und jede dieser Tätigkeiten hinterlässt ein nachvollziehbares Ergebnis.

1.1 Strukturelle und funktionale Qualität

- **Funktionale Qualität:** Tut die Software das Richtige? Merkmale: Korrektheit, Zuverlässigkeit, Sicherheit, Performanz. Sichtbar für den Nutzer, prüfbar durch Tests gegen Anforderungen.
- **Strukturelle Qualität:** Ist die Software gut gebaut? Merkmale: Wartbarkeit, Verständlichkeit, Testbarkeit, Wiederverwendbarkeit. Unsichtbar für den Nutzer, prüfbar durch Metriken, Reviews und statische Analyse.

Der Unterschied ist prüfungsrelevant: funktionale Qualität wird gegen die Spezifikation getestet, strukturelle Qualität gegen Standards und Metriken gemessen.

Hinweis

Schlechte strukturelle Qualität kostet nicht sofort. Sie kostet bei jeder Änderung, und der Effekt ist kumulativ. Genau deshalb braucht sie automatisierte Messung: sie meldet sich nicht von allein.

2 Anforderungen und Spezifikation

2.1 Arten von Anforderungen

- **Funktionale Anforderungen:** Was das System tun soll. „Das System berechnet den Rabatt aus Warenkorbwert und Kundenstatus.“
- **Nichtfunktionale Anforderungen (Qualitätsattribute):** Wie gut es das tun soll. Antwortzeit, Verfügbarkeit, Skalierbarkeit, Sicherheit.
- **Randbedingungen (Constraints):** Vorgaben, die nicht verhandelbar sind. Zielplattform, gesetzliche Auflagen, vorhandene Systeme.

Nichtfunktionale Anforderungen treiben die Architektur; funktionale Anforderungen treiben die Implementierung. Deshalb beginnt ADD (Abschnitt ??) mit den Qualitätsattributen und nicht mit den Use Cases.

2.2 Gute Anforderungen

Eine Anforderung ist brauchbar, wenn sie **überprüfbar** ist. Die übliche Checkliste:

Kriterium	Gegenbeispiel
eindeutig	„Das System soll schnell sein.“
vollständig	„Fehler werden behandelt.“ (welche? wie?)
widerspruchsfrei	zwei Anforderungen fordern Gegenteiliges
überprüfbar	„benutzerfreundlich“ ohne Messvorschrift
verfolgbar	Anforderung ohne Quelle und ohne Test

Aus „schnell“ wird überprüfbar: *95 % der Suchanfragen werden in unter 200 ms beantwortet, gemessen serverseitig bei 100 gleichzeitigen Nutzern.*

2.3 User Stories und Use Cases

User Story: *Als <Rolle> möchte ich <Ziel>, um <Nutzen>.* Kurz, verhandelbar, mit Akzeptanzkriterien. Sie ist eine Erinnerung an ein Gespräch, keine Spezifikation.

Use Case: Ablauf mit Akteur, Vorbedingung, Standardablauf, Alternativabläufen und Nachbedingung. Deutlich formaler und geeignet, wenn das Zusammenspiel vieler Schritte das Schwierige ist.

2.4 Logik: Erfüllbarkeit, Widerspruch, Implikation

Anforderungen lassen sich als aussagenlogische Formeln über Booleschen Variablen schreiben. Drei Begriffe, die in der Verifikation wiederkehren:

Definition

Seien φ, ψ Formeln in KNF.

- **Erfüllbarkeit (SAT):** φ ist erfüllbar, wenn eine Belegung v existiert mit $\beta[\varphi, v] = 1$.
- **Widerspruch:** φ und ψ sind widersprüchlich, wenn $(\varphi \wedge \psi)$ nicht erfüllbar ist.
- **Implikation:** $\varphi \models \psi$ gilt, wenn $(\varphi \wedge \neg\psi)$ nicht erfüllbar ist.

Hinweis

Der letzte Punkt ist der Schlüssel zum gesamten Verifikationsteil der Vorlesung. Jede Implikationsfrage wird beantwortet, indem man ein Modell der Negation sucht. Findet man eines, ist es ein Gegenbeispiel; findet man keines, gilt die Implikation.

3 Vorgehensmodelle und Prozessreife

3.1 Wasserfall und V-Modell

Das **Wasserfallmodell** ordnet die Phasen streng sequenziell: Anforderungen, Entwurf, Implementierung, Test, Betrieb. Jede Phase endet mit einem Dokument, und Rücksprünge sind teuer.

Das **V-Modell** biegt den Wasserfall zu einem V und ordnet jeder konstruktiven Phase eine Teststufe zu.

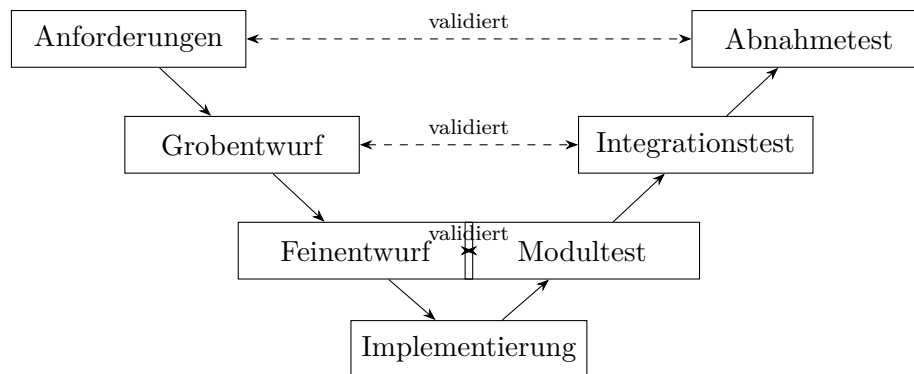


Abbildung 1: V-Modell: jede Entwurfsstufe hat ihre Teststufe

Die waagerechten Linien sind der eigentliche Inhalt: der Abnahmetest prüft gegen die Anforderungen, der Integrationstest gegen den Grobentwurf, der Modultest gegen den Feinentwurf. Ein Test ohne zugehöriges Dokument prüft gegen nichts.

3.2 Agiles Vorgehen und Scrum

Scrum ersetzt die lange Phasenkette durch kurze, vollständige Zyklen.

Rollen	Artefakte	Ereignisse
Product Owner	Product Backlog	Sprint Planning
Scrum Master	Sprint Backlog	Daily Scrum
Entwicklungsteam	Increment	Sprint Review
	Definition of Done	Sprint Retrospective

Der Product Owner verantwortet *was*, das Team verantwortet *wie*, der Scrum Master verantwortet, dass der Prozess funktioniert. Diese Trennung ist der häufigste Prüfungspunkt.

Hinweis

Scrum ist kein Vorgehensmodell für Entwurf, sondern für Steuerung. Was gebaut wird, entscheidet der Backlog; *wie* es strukturiert wird, sagt Scrum nicht. Deshalb schließen sich Scrum und Architekturarbeit nicht aus.

3.3 CMMI: Prozessreife

CMMI misst nicht die Software, sondern den *Prozess*, der sie herstellt. Fünf Reifegrade:

Grad	Name	Kennzeichen
1	Initial	Erfolg hängt an einzelnen Personen, Prozesse ad hoc
2	Managed	Projekte werden geplant, verfolgt und gesteuert
3	Defined	Ein organisationsweiter Standardprozess existiert
4	Quantitatively Managed	Der Prozess wird quantitativ gemessen und gesteuert
5	Optimizing	Der Prozess wird auf Basis der Messung systematisch verbessert

Der Sprung von 3 auf 4 ist der interessante: ab dort wird der Prozess mit Zahlen gesteuert, was ohne Grad 3 nicht geht, weil es vorher keinen gemeinsamen Prozess gibt, über den man Zahlen bilden könnte.

4 Schätzen und Messen

4.1 Zwei- und Dreipunktschätzung

Der Aufwand eines Arbeitspakets ist eine Zufallsgröße mit unbekannter Dichte. Man schätzt drei Werte und *nimmt eine Verteilung an*:

Definition

Mit kleinstem Wert a , mittlerem Wert c und größtem Wert b :

$$E(x) = \frac{a + r \cdot c + b}{2 + r}, \quad V(x) = \frac{(b - a)^2}{u^2}, \quad S(x) = \frac{b - a}{u}$$

Annahme	r	u
Zweipunktschätzung	0	6
PERT (Beta-Näherung)	4	6
Beta ($p = q = 3$, quadratisch)	4	5,29
Beta ($p = q = 4$, kubisch)	6	6
Beta ($p = 4, q = 2$)	4	5,6

Damit ist die Zweipunktschätzung $E = \frac{a+b}{2}$ und die Dreipunktschätzung $E = \frac{a+4c+b}{6}$, beide mit $S = \frac{b-a}{6}$.

4.2 Aggregation über Arbeitspakete

$$E = \sum_i E_i, \quad S = \sqrt{\sum_i S_i^2}$$

Erwartungswerte addieren sich, Standardabweichungen *nicht*: es addieren sich die Varianzen. Fünf Pakete mit je $S_i = 6$ ergeben zusammen $S = 13,4$ und nicht 30.

Hinweis

Das gilt nur bei **unabhängigen** Paketen. Ein gemeinsamer Engpass, ein gemeinsamer Zulieferer oder eine Person, die in allen Paketen steckt, zerstört die Annahme, und keine Formel bemerkt das.

Beispiel: Spieleprojekt (Übungsblatt 2)

Komponente	<i>a</i>	<i>c</i>	<i>b</i>	<i>E</i> (3-Pkt.)	<i>S</i>
Datenverwaltung	8	20	25	18,83	2,83
Gamelogik	30	64	128	69,00	16,33
Grafikengine	42	121	180	117,67	23,00
Gamepad-Interface	8	16	20	15,33	2,00
Security	10	30	45	29,17	5,83
Summe				250,00	29,01

Zweipunktschätzung: $E = 248,00$ bei identischem $S = 29,01$. Das ist kein Zufall: S hängt nur von der Spannweite ab, nicht vom mittleren Wert.

4.3 Velocity**Definition**

Velocity ist die Anzahl Story Points, die ein Team pro Sprint *fertigstellt*. Story Points messen Umfang, nicht Zeit; Velocity ist die einzige Brücke zwischen beiden, und sie wird gemessen, nicht geschätzt.

Regeln aus der Vorlesung:

- Nur **fertige** Stories zählen. Halb fertige Arbeit bringt null Punkte.
- Nach etwa drei Sprints pendelt sich die Velocity ein.
- Danach plant man mit dem **Median** der letzten Sprints, nicht mit dem Mittelwert: der Median ist robust gegen einen einzelnen schlechten Sprint.

Beispiel (Übungsblatt 2, Aufgabe 2.2)

Sprint	geplant	tatsächlich	fertiggestellt
1	15	11	U2, U3, U4
2	16	11	U1, U5, U7
3	15	10	U6, U8
4	9	5	U9
5	4	4	U10

Median der letzten drei Sprints: 5. Insgesamt wurden 41 von 59 zugesagten Punkten geliefert, das Team übernimmt sich also systematisch. Zwei typische Ursachen für sinkende Velocity: wachsende technische Schulden und Aufgaben, die nicht im Sprint Backlog stehen (Support, Störungen).

4.4 Planning Poker

Alle schätzen gleichzeitig, die Extreme begründen sich, es wird neu geschätzt. Das Ergebnis ist der Konsens, das Produkt ist die Diskussion.

Die Kartenfolge ist bewusst nicht linear: 1, 2, 3, 5, 8, 13, 21. Große Stories lassen sich nicht genauer schätzen als kleine, und die wachsenden Abstände sagen das ehrlich, statt eine Unterscheidung zwischen 17 und 18 anzubieten.

Triangulation: Schätzungen werden gegeneinander geprüft, nicht gegen Stunden. Ist die 2-Punkte-Story wirklich doppelt so groß wie die 1-Punkte-Story? Sind beide 5-Punkte-Stories ähnlich groß?

4.5 Messen im Betrieb: GQM und KPI

Definition

GQM (Goal – Question – Metric) nach Basili und Rombach: erst das Ziel, dann die Fragen, die man beantworten muss, und erst dann die Metriken, die diese Fragen beantworten. Nie in der umgekehrten Richtung.

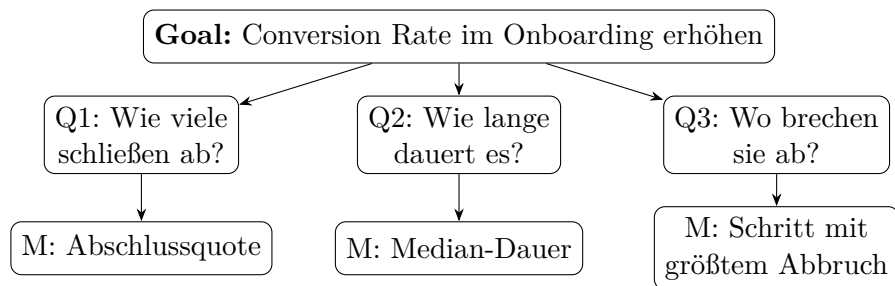


Abbildung 2: GQM-Baum: die Pfeilrichtung ist die Methode

Ein **KPI** ist eine Kennzahl, an die eine *Entscheidung* gekoppelt ist. Eine Kennzahl, bei der niemand anders handeln würde, ist Dekoration.

Hinweis

Goodharts Gesetz: Sobald eine Kennzahl zum Ziel wird, hört sie auf, ein gutes Maß zu sein. Typisches Muster: die Kennzahl steigt stetig, das eigentliche Ergebnis bewegt sich nicht.

4.6 A/B-Tests

Monitoring sagt, *was* passiert ist. Ein Experiment sagt, ob die eigene Änderung es *verursacht* hat.

Der Stichprobenumfang wird **vorher** bestimmt, aus Basisrate, kleinstem relevanten Effekt, Signifikanzniveau α und Power. Bei Basisrate 10 %:

Zu erkennender Effekt	Nutzer je Gruppe
+20 %	3 841
+10 %	14 751
+5 %	57 763

Halbiert man den Effekt, vervierfacht sich die Stichprobe.

Hinweis

Der p-Wert ist nicht die Wahrscheinlichkeit, dass die Änderung wirkt.
Er sagt: wenn die Änderung nichts bewirkt, wie oft erzeugt der Zufall allein einen so großen Unterschied.

Peeking: Wer den laufenden Test wiederholt anschaut und beim ersten signifikanten Ergebnis abbricht, erhöht die Fehlerrate drastisch. In der Simulation mit zwei *identischen* Gruppen: 5 % falsche Signifikanzen bei einmaliger Auswertung am Ende, 17 % bei zehnmaligem Hineinschauen.

5 Architektur und Modellierung

5.1 Kopplung und Kohäsion

Definition

Kopplung ist das Maß dafür, wie stark ein Element mit anderen verbunden ist, Wissen über sie besitzt oder von ihnen abhängt.

Kohäsion ist das Maß dafür, wie stark zusammenhängend die Verantwortlichkeiten innerhalb eines Elements sind.

Die Entwurfsregel lautet: **Kopplung minimieren, Kohäsion maximieren.** Beides zugleich ist oft ein Zielkonflikt, denn Verantwortlichkeiten zusammenzuziehen erhöht die Kohäsion und schafft neue Beziehungen nach außen.

Typische Formen der Kopplung zwischen *A* und *B*: *A* hat ein Attribut vom Typ *B*, *A* ruft einen Dienst von *B* auf, *A* hat eine Methode mit *B* als Parameter, Rückgabewert oder lokaler Variable, *A* erbt von *B*, *A* implementiert die Schnittstelle *B*.

Folgen starker Kopplung: lokale Änderungen ziehen Änderungen in verbundenen Klassen nach sich, Klassen sind einzeln schwer zu verstehen und schwer wiederzuverwenden. Folgen geringer Kohäsion: unklare Verantwortung, schlechte Verständlichkeit, schlechtere Wartbarkeit.

5.2 LCOM

Definition

Für eine Klasse mit a Feldern und m Methoden sei $n(A_i)$ die Zahl der Methoden, die auf Feld A_i zugreifen. Dann ist

$$LCOM = \frac{\left(\frac{1}{a} \sum_{i=1}^a n(A_i)\right) - m}{1 - m}$$

mit Wertebereich $[0, 1]$. 0 bedeutet hohe Kohäsion (jede Methode nutzt jedes Feld), 1 bedeutet sehr geringe Kohäsion.

Beispiele

Dreieck: $a = 3$, $m = 4$, jedes Feld von jeder Methode genutzt.

$$LCOM = \frac{\frac{1}{3} \cdot 12 - 4}{1 - 4} = \frac{4 - 4}{-3} = 0$$

Figur (Dreieck und Kreis verschmolzen): $a = 4$, $m = 8$, jedes Feld weiterhin von nur vier Methoden genutzt.

$$LCOM = \frac{\frac{1}{4} \cdot 16 - 8}{1 - 8} = \frac{4 - 8}{-7} = \frac{4}{7} \approx 0,57$$

Übungsaufgabe: A_1, A_2, A_3 und $m_1 \dots m_4$ mit $m_1 : \{A_1, A_2\}$, $m_2 : \{A_2\}$, $m_3 : \{A_2, A_3\}$, $m_4 : \{A_3\}$. Also $n(A_1) = 1$, $n(A_2) = 3$, $n(A_3) = 2$, Summe 6:

$$LCOM = \frac{\frac{1}{3} \cdot 6 - 4}{1 - 4} = \frac{2 - 4}{-3} = \frac{2}{3} \approx 0,67$$

Das ist weit von 0 entfernt und deutet auf mittlere bis geringe Kohäsion hin.

5.3 Sichten

Eine Architektur wird nicht in einem Bild dargestellt, sondern in mehreren **Sichten**, weil verschiedene Fragen verschiedene Strukturen betreffen:

Sicht	Beantwortet
Baustein / Modul	Woraus besteht das System, wer hängt von wem ab?
Laufzeit	Welche Prozesse und Objekte existieren zur Laufzeit, wie kommunizieren sie?
Verteilung	Auf welcher Hardware läuft was?

5.4 SOLID

S	Single Responsibility	Eine Klasse hat genau einen Grund, sich zu ändern
O	Open/Closed	Offen für Erweiterung, geschlossen für Änderung
L	Liskov Substitution	Untertypen müssen ohne Bruch an die Stelle des Obertyps treten können
I	Interface Segregation	Viele kleine Schnittstellen statt einer großen
D	Dependency Inversion	Abhängigkeit auf Abstraktionen, nicht auf Implementierungen

Single Responsibility ist die Kohäsionsregel aus Abschnitt ?? in Worten, Dependency Inversion die Kopplungsregel: Abhängigkeiten sollen von instabilen zu stabilen Bausteinen zeigen.

5.5 ADD 3.0: Attribute Driven Design

ADD ist ein *iteratives* Entwurfsverfahren, das von Qualitätsattributen ausgeht und nicht von Funktionen. Sieben Schritte je Iteration:

1. **Eingaben prüfen** – Ziele, Use Cases, Qualitätsszenarien, Randbedingungen und Concerns sichten und priorisieren.
2. **Iterationsziel festlegen** – welches Designziel diese Iteration verfolgt, etwa „Primärfunktion strukturieren“ oder „Latenzproblem adressieren“.
3. **Zu verfeinernde Elemente wählen** – welche Systemteile bearbeitet werden, typischerweise vom Groben zum Detail.
4. **Designkonzepte auswählen** – Muster und Taktiken identifizieren und gegeneinander abwägen.
5. **Elemente instanziierten, Verantwortungen zuordnen, Schnittstellen definieren** – konkrete Bausteine samt Protokollen und Datenflüssen.
6. **Sichten skizzieren und Entscheidungen protokollieren** – inklusive Begründung und verworfener Alternativen.
7. **Analyse und Review gegen das Iterationsziel** – Abdeckung, Restrisiken, Aufgaben für die nächste Iteration.

Iterativ heißt: nicht die ganze Architektur auf einmal, sondern je Runde ein priorisiertes Ziel. Vorteile: frühe Rückmeldung, beherrschbare Entscheidungen und die Möglichkeit, bei geänderten Treibern die Richtung zu wechseln.

6 Automation

6.1 Build und Abhängigkeiten

Ein Build-Werkzeug (Maven, Gradle, npm) macht drei Dinge reproduzierbar: die **Abhängigkeiten** mit ihren Versionen, die **Schritte** vom Quelltext zum Artefakt und die **Umgebung**, in der das geschieht.

Transitive Abhängigkeiten sind der schwierige Teil: A braucht B, B braucht C in Version 1, D braucht C in Version 2. Version-Locking und Konfliktauflösung sind deshalb Kernfunktionen des Werkzeugs und keine Nebensache.

6.2 Continuous Integration und Delivery

- **Continuous Integration:** Jede Änderung wird zeitnah integriert und automatisch gebaut und getestet. Ziel ist, Integrationsfehler früh und in kleinen Portionen zu finden.
- **Continuous Delivery:** Jeder erfolgreiche Build ist prinzipiell auslieferbar. Die Freigabe bleibt eine Entscheidung.
- **Continuous Deployment:** Jeder erfolgreiche Build wird automatisch ausgeliefert.

Eine Pipeline ordnet die Prüfungen nach Kosten: erst Kompilieren und Modultests, dann Integrationstests, dann langsame Prüfungen wie Last- oder Sicherheitstests. Wer teuer vor billig prüft, bezahlt jede triviale Regression mit voller Laufzeit.

7 Qualitätssicherung und Programmanalyse

7.1 Einordnung der Verfahren

	statisch (ohne Ausführung)	dynamisch (mit Ausführung)
prüfend	Review, Coding Standards, Metriken	Test, Überdeckungsmessung
beweisend	Programmanalyse, Model Checking, Hoare	–

Ein Test zeigt die Anwesenheit von Fehlern, nie deren Abwesenheit. Ein Beweis zeigt die Abwesenheit, kostet aber eine Spezifikation und einen Aufwand, den man sich gezielt aussuchen muss.

7.2 Metriken

- **LOC:** Zeilen Code. Einfachster Indikator, misst Tippen und nicht Schwierigkeit.
- **LCOM:** fehlende Kohäsion, siehe Abschnitt ??.
- **CC:** zyklomatische Komplexität, siehe unten.

Ein **Coding Standard** ist erst dann ein Standard, wenn seine Regeln maschinell prüfbar sind: ein Build kann daran scheitern. Werkzeuge wie PMD suchen dafür Muster im abstrakten Syntaxbaum.

7.3 Die Sprache While

$$S ::= [x := a]^l \mid [\text{skip}]^l \mid S_1; S_2 \mid \text{if } [b]^l \text{ then } S_1 \text{ else } S_2 \mid \text{while } [b]^l \text{ do } S$$

Labels identifizieren Anweisungen **eindeutig**. Vier Funktionen darauf:

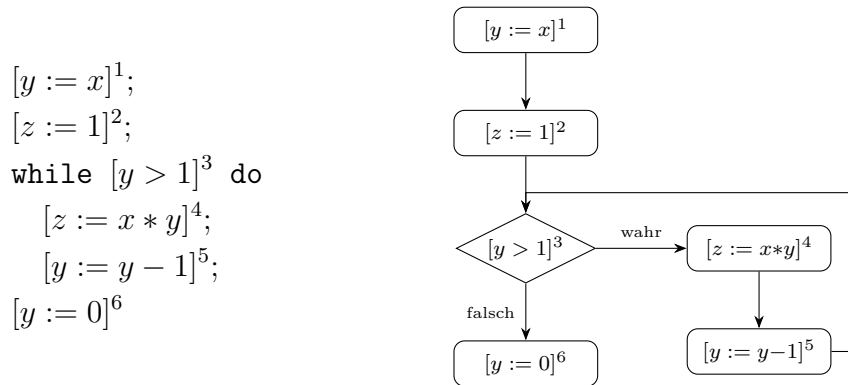
$$\begin{aligned}
 init([x := a]^l) &= l & final([x := a]^l) &= \{l\} \\
 init(S_1; S_2) &= init(S_1) & final(S_1; S_2) &= final(S_2) \\
 init(\text{if } [b]^l \dots) &= l & final(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) &= final(S_1) \cup final(S_2) \\
 init(\text{while } [b]^l \dots) &= l & final(\text{while } [b]^l \text{ do } S) &= \{l\}
 \end{aligned}$$

$$\begin{aligned}
 flow(S_1; S_2) &= flow(S_1) \cup flow(S_2) \cup \{(l, init(S_2)) : l \in final(S_1)\} \\
 flow(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) &= flow(S_1) \cup flow(S_2) \cup \{(l, init(S_1)), (l, init(S_2))\} \\
 flow(\text{while } [b]^l \text{ do } S) &= flow(S) \cup \{(l, init(S))\} \cup \{(l', l) : l' \in final(S)\}
 \end{aligned}$$

Definition

Der **Kontrollflussgraph** eines Programms S ist (V, E) mit $V = labels(S)$ und $E = flow(S)$.

Beispiel



$$flow = \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 3), (3, 6)\}$$

7.4 Zyklomatische Komplexität

Definition

$CC = e - n + 2p$ mit e Kanten, n Knoten und p Zusammenhangskomponenten des Kontrollflussgraphen. CC ist die Zahl der unabhängigen Pfade.

Für das Beispiel: $6 - 6 + 2 \cdot 1 = 2$. Geradliniger Code hat $CC = 1$, jede Verzweigung erhöht den Wert um eins. CC ist eine untere Schranke für die Zahl der Testfälle, die man für Zweigüberdeckung braucht.

7.5 Reaching Definitions

Definition

Ein Paar (x, l) bedeutet: die Zuweisung an Label l kann hier noch der aktuelle Wert von x sein. $(x, ?)$ bedeutet: x ist möglicherweise nicht initialisiert.

$$kill_{RD}([x := a]^l) = \{(x, ?)\} \cup \{(x, l') : [B]^{l'} \text{ weist } x \text{ zu}\}$$

$$gen_{RD}([x := a]^l) = \{(x, l)\}$$

$$kill_{RD}([\text{skip}]^l) = kill_{RD}([b]^l) = \emptyset, \quad gen_{RD}([\text{skip}]^l) = gen_{RD}([b]^l) = \emptyset$$

$$RD_{entry}(l) = \begin{cases} \{(x, ?) : x \in FV(S)\} & \text{für } l = init(S) \\ \bigcup \{RD_{exit}(l') : (l', l) \in flow(S)\} & \text{sonst} \end{cases}$$

$$RD_{exit}(l) = (RD_{entry}(l) \setminus kill_{RD}([B]^l)) \cup gen_{RD}([B]^l)$$

Die Analyse ist **vorwärts** und eine **may**-Analyse: vereinigt wird, weil eine Definition ein Label erreicht, wenn sie es auf *irgendeinem* Pfad erreicht. Anwendung: Ist $(x, ?)$ am Eingang eines Labels, das x liest, liegt möglicherweise eine Benutzung vor der Initialisierung vor.

7.6 Die vier klassischen Datenflussanalysen

	may (Vereinigung)	must (Schnitt)
vorwärts	Reaching Definitions	Available Expressions
rückwärts	Live Variables	Very Busy Expressions

Alle vier sind derselbe Algorithmus mit vier Stellschrauben: Richtung, Verknüpfung an Zusammenflüssen, Startwert und die Funktionen $kill$ und gen . Die Transferfunktion ist immer $(\text{eingehend} \setminus kill) \cup gen$.

7.7 Strukturierte operationelle Semantik

$\langle \text{skip}, v \rangle \Rightarrow v$	[SKIP]
$\langle x := a, v \rangle \Rightarrow v[x \mapsto c], \quad c = \alpha[a, v]$	[ASS]
$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_1, v \rangle, \text{ falls } \beta[b, v] = 1$	[IF _T]
$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_2, v \rangle, \text{ falls } \beta[b, v] = 0$	[IF _F]
$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow \langle S; \text{while } b \text{ do } S, v \rangle, \text{ falls } \beta[b, v] = 1$	[WH _T]
$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow v, \text{ falls } \beta[b, v] = 0$	[WH _F]
$\frac{\langle S_1, v \rangle \Rightarrow \langle S'_1, v' \rangle}{\langle S_1; S_2, v \rangle \Rightarrow \langle S'_1; S_2, v' \rangle}$	[COMP ₁]
$\frac{\langle S_1, v \rangle \Rightarrow v'}{\langle S_1; S_2, v \rangle \Rightarrow \langle S_2, v' \rangle}$	[COMP ₂]

Zwei Axiome überführen eine Konfiguration in einen *Zustand* (SKIP, ASS, WH_F), die übrigen in eine *Konfiguration*. Die Schleife wird pro Schritt einmal abgerollt; deshalb beschreibt ein endliches Regelsystem eine unbeschränkte Ausführung.

7.8 Symbolische Ausführung

Dieselben Regeln, aber die Variablen zeigen auf *Ausdrücke über symbolischen Werten*, und jeder Zweig trägt eine **Pfadbedingung** φ :

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v, \varphi \rangle \Rightarrow \langle S_1, v, \varphi \wedge \sigma[b, v] \rangle \quad \text{falls erfüllbar}$$

Beide Regeln können gelten, es entsteht ein **Ausführungsbaum**. Ein Zweig mit unerfüllbarer Pfadbedingung wird verworfen; das Lösen einer Pfadbedingung liefert konkrete Eingaben, die genau dieses Blatt erreichen – also automatisch erzeugte Testfälle.

Grenze: jede Schleifeniteration ist eine weitere Abrollung. Bei unbeschränkter Iterationszahl ist der Baum unendlich, und eine Tiefenschränke ist die praktische Antwort, keine Lösung.

8 Verifikation

8.1 DPLL

Entscheidet die Erfüllbarkeit einer Formel in KNF. Zwei Beobachtungen sparen ganze Teilbäume:

- **Unit-Klausel:** Bleibt in einer Klausel genau ein unbelegtes Literal, ist seine Belegung erzwungen.
- **Pures Literal:** Kommt eine Variable nur mit einem Vorzeichen vor, kann sie entsprechend belegt werden, ohne je zu schaden.

DPLL(C, v):

```

 $C^- \leftarrow \{c \in C : c[v] \rightarrow false\};$  if  $C^-$  nicht leer then return false
 $C^* \leftarrow C \setminus \{c \in C : true \rightarrow c[v]\};$  if  $C^*$  leer then return true
if Unit-Klausel für  $x$  oder  $\bar{x}$  then return DPLL( $C^*, v[x \mapsto b]$ )
if pures Literal  $x$  oder  $\bar{x}$  then return DPLL( $C^*, v[x \mapsto b]$ )
wähle  $x$ ; return DPLL( $C^*, v[x \mapsto 0]$ )  $\vee$  DPLL( $C^*, v[x \mapsto 1]$ )

```

Beispiel

$$(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

1. x_1 ist pur $\Rightarrow v[x_1 \mapsto 1]$, erste Klausel erfüllt
2. Verzweigung $v[x_2 \mapsto 0]$
3. $\neg x_3$ ist Unit $\Rightarrow v[x_3 \mapsto 0]$, C^* leer

Eine geprüfte Belegung statt $2^3 = 8$. Der Worst Case bleibt $O(2^n)$: SAT ist NP-vollständig, und keine der beiden Regeln ändert daran etwas.

8.2 Transitionssysteme

Definition

Ein **Transitionssystem** ist ein Tripel $T = \langle s, I, \Gamma \rangle$ mit einer Menge getypter Variablen s , einem Booleschen Ausdruck I über s für die Anfangszustände und einem Ausdruck Γ über $s \cup s'$ für die Transitionsrelation.

Eine Komponente hat Eingaben und Ausgaben. Komponiert man sie mit einem **Treiber**, der Eingaben liefert, und einem **Monitor**, der Ausgaben prüft, entsteht ein geschlossenes System ohne Ein- und Ausgaben – genau ein Transitionssystem. Komposition ist Konjunktion der Relationen; gemeinsame Namen sind die Verdrahtung.

8.3 Bounded Model Checking

Definition

Mit $\Gamma_{i,j} = \Gamma[s_i/s][s_j/s']$ und $P^k = P[s_k/s]$ heißt T **k -sicher** für P , wenn

$$(I[s_0/s] \wedge \Gamma_{0,1} \wedge \cdots \wedge \Gamma_{k-1,k}) \models (P^0 \wedge \cdots \wedge P^k)$$

Getestet wird wie jede Implikation, über ein Modell der Negation:

$$I[s_0/s] \wedge \Gamma_{0,1} \wedge \cdots \wedge \Gamma_{k-1,k} \wedge (\neg P^0 \vee \cdots \vee \neg P^k)$$

Algorithmus: $k = 0$; ist der Test erfüllbar, ist ein Fehler gefunden und das Modell ist das Gegenbeispiel; sonst $k \leftarrow k + 1$ und erneut.

Beispiel (Klausuraufgabe)

$T = \langle \{s_1\}, s_1 = 20, s'_1 = s_1/2, P = (s_1 \bmod 2 = 0) \rangle$.

$$\Gamma_{0,1} : s_1^{(1)} = s_1^{(0)}/2, \quad \Gamma_{1,2} : s_1^{(2)} = s_1^{(1)}/2, \quad \Gamma_{2,3} : s_1^{(3)} = s_1^{(2)}/2$$

Der Lauf ist $20 \rightarrow 10 \rightarrow 5 \rightarrow 2$. Für $k = 0$ und $k = 1$ ist das System k -sicher; für $k = 2$ ist der Test erfüllbar, das Gegenbeispiel ist 20, 10, 5 und verletzt P im dritten Zustand.

Hinweis

BMC beweist **nie** Sicherheit. Die Menge der in i Schritten erreichbaren Zustände wächst mit i , und es gibt keine Formel, die sagt, wann sie aufhört zu wachsen. Genau diese Lücke schließen induktive Invarianten.

8.4 Induktive Invarianten

Definition

P ist eine **induktive Invariante** von $T = \langle s, I, \Gamma \rangle$, wenn

1. **Basis:** $I \models P$
2. **Schritt:** $P \wedge \Gamma \models P[s'/s]$

Gelten beide, so gilt P in jedem erreichbaren Zustand, per Induktion über die Länge des Laufs – ohne jede Schranke.

Der übliche Stolperstein: Der Schrittbeweis startet in *jedem* Zustand, der P erfüllt, auch in unerreichbaren. Beispiel: $x' = x + 2$ mit $I : x = 0$ und $P : x \neq 5$. P gilt in allen erreichbaren Zuständen (sie sind gerade), ist aber nicht induktiv: aus $x = 3$ folgt $x' = 5$. $x = 3$ ist unerreichbar, und der Schrittest weiß das nicht.

Verstärkung: Man sucht ein stärkeres Q , das P impliziert und induktiv ist. Hier genügt $x \bmod 2 = 0$, und $P \wedge (x \bmod 2 = 0)$ ist induktiv.

8.5 Hoare-Logik

Definition

Das Tripel $\{P\} S \{Q\}$ bedeutet: gilt P vor S und terminiert S , so gilt Q danach (*partielle Korrektheit*).

$\frac{}{\{Q\} \text{ skip } \{Q\}}$	Skip
$\frac{}{\{Q[a/x]\} x := a \{Q\}}$	Zuweisung
$\frac{\{P\} S_1 \{R\}, \{R\} S_2 \{Q\}}{\{P\} S_1; S_2 \{Q\}}$	Komposition
$\frac{\{P \wedge b\} S_1 \{Q\}, \{P \wedge \neg b\} S_2 \{Q\}}{\{P\} \text{ if } b \text{ then } S_1 \text{ else } S_2 \{Q\}}$	Verzweigung
$\frac{\{I \wedge b\} S \{I\}}{\{I\} \text{ while } b \text{ do } S \{I \wedge \neg b\}}$	Schleife
$\frac{P \rightarrow P', \{P'\} S \{Q'\}, Q' \rightarrow Q}{\{P\} S \{Q\}}$	Konsequenz

Die Zuweisungsregel läuft **rückwärts**: um zu wissen, was vor $x := a$ gelten muss, ersetzt man x in Q durch a . Vorwärts wäre es geraten, rückwärts ist es eine Gleichung.

Die Schleife ist die einzige Regel mit echter Arbeit: sie braucht eine **Invariante**, die keine Regel liefert.

Beispiel (Übungsblatt 6)

```

{ true }
if (a >= b) { while (a != b) a := a - 1; }
else { while (a != b) a := a + 1; }
{ a = b }

```

Invarianten: im **then**-Zweig $a \geq b$, im **else**-Zweig $a \leq b$. Nachweis für den **then**-Zweig:

aus $a \geq b \wedge a \neq b$ folgt $a > b$, also $a - 1 \geq b$, die Invariante bleibt erhalten. Beim Verlassen gilt $a \geq b \wedge \neg(a \neq b)$, also $a = b$.

9 Modellbasiertes Testen

9.1 Mealy-Automaten

Ein Mealy-Automat antwortet auf jedes Eingabesymbol mit einem Ausgabesymbol; ein Wort der Länge n erzeugt ein Ausgabewort der Länge n . Damit ist er das natürliche Modell einer reaktiven Komponente.

Zustand	bei a	bei b
q_0	$q_1/0$	$q_2/1$
q_1	$q_1/1$	$q_2/0$
q_2	$q_0/0$	$q_2/1$

Für M_1 : $Out(abba) = 0010$ und $Out(baab) = 1000$. Ein **Testorakel** vergleicht die behauptete Ausgabe eines Systems mit dieser Vorhersage.

9.2 Überdeckungen

Überdeckung	Idee	Größe für M_1
Zustandsüberdeckung	jeden Zustand erreichen	3
Transitionsüberdeckung	jede Transition ausführen	6
Transition-Switch	jedes ausführbare Transitionspar	12
W-Methode	Transitionsüberdeckung + Unterscheidungsmenge	vollständig unter An

Bei n Zuständen und k Eingabesymbolen gibt es $n \cdot k \cdot k$ Paare direkt aufeinanderfolgender Transitionen, für M_1 also $3 \cdot 2 \cdot 2 = 12$. Die Suite baut man systematisch über Zugriffssequenzen $\alpha(q)$:

$$T = \{\alpha(q) xy : q \in Q, x, y \in \Sigma\}$$

mit $\alpha(q_0) = \varepsilon$, $\alpha(q_1) = a$, $\alpha(q_2) = b$.

9.3 Observation Table und L^*

Ein Lerner sieht das System nicht und darf zwei Fragen stellen: **Membership Query** (führe dieses Wort aus, was kommt zurück?) und **Equivalence Query** (stimmt diese Hypothese, und wenn nicht, wo weicht sie ab?).

Definition

Die **Observation Table** hat Zeilen U (Zugriffswörter) und $U \cdot A$ (Rand), Spalten V (Suffixe) und in jeder Zelle das *letzte* Ausgabesymbol von $u \cdot v$.

- **closed**: jede Zeile aus $U \cdot A$ kommt bereits in U vor
- **consistent**: gleich aussehende Zeilen bleiben nach einem weiteren Symbol gleich

Reparatur: nicht geschlossen $\Rightarrow$ Zeile nach U verschieben (neuer Zustand); nicht konsistent $\Rightarrow$ Spalte hinzufügen (neues Experiment).

Ablauf von L*: Tabelle füllen, schließen, konsistent machen, Hypothese bauen, Äquivalenz anfragen; bei Gegenbeispiel Tabelle verfeinern und wiederholen. Jede Runde fügt mindestens einen Zustand hinzu, das Ziel hat endlich viele, also terminiert es.

Hinweis

Die Equivalence Query beantwortet in der Praxis niemand. Sie wird durch eine Testsuite angenähert – und damit ist das gelernte Modell nur so gut wie diese Suite. Lernen liefert die Tests, die Tests liefern das Lernen: dieser Kreis ist der tatsächliche Stand des Verfahrens, kein Darstellungsfehler.

10 Formelsammlung

Thema	Formel
Zyklomatische Komplexität	$CC = e - n + 2p$
LCOM	$\left(\frac{1}{a} \sum n(A_i) - m\right) / (1 - m)$, Bereich $[0, 1]$
Punktschätzung	$E = \frac{a+rc+b}{2+r}$, $S = \frac{b-a}{u}$; 2-Pkt.: $r = 0$, 3-Pkt./PERT: $r = 4$, $u = 6$
Aggregation	$E = \sum E_i$, $S = \sqrt{\sum S_i^2}$
Velocity	Story Points fertiger Stories pro Sprint; Planung mit Median
Implikation	$\varphi \models \psi$ gdw. $\varphi \wedge \neg\psi$ unerfüllbar
k -Sicherheit	$I[s_0] \wedge \Gamma_{0,1} \wedge \dots \wedge \Gamma_{k-1,k} \models P^0 \wedge \dots \wedge P^k$
Induktive Invariante	Basis $I \models P$; Schritt $P \wedge \Gamma \models P[s'/s]$
Hoare-Zuweisung	$\{Q[a/x]\} x := a \{Q\}$
Reaching Definitions	$RD_{exit}(l) = (RD_{entry}(l) \setminus kill) \cup gen$
Transitionspaare	$n \cdot k^2$ bei n Zuständen und k Eingabesymbolen

Hinweis

Zu jedem Kapitel dieser Notizen gibt es lauffähigen Code im Verzeichnis `swk/`: `program-analysis` für While, CFG und Datenfluss, `verification` für DPLL, BMC, Invarianten und Hoare, `model-based-testing` für Mealy, L* und Überdeckungen, `metrics` für LCOM und Kopplung, `estimation` für Schätzung, Velocity und A/B-Tests. Alle Beispiele dieser Notizen sind dort nachgerechnet.